

SERVIÇO NACIONAL DE APRENDIZAGEM INDUSTRIAL – SENAI
CURSO TÉCNICO EM DESENVOLVIMENTO DE SISTEMA

ERICK SILVA FERNANDES DE ARAÚJO

DOCUMENTAÇÃO INDIVIDUAL SOBRE OS TESTES UNITÁRIOS
Iveco Green Ledger - Sistema de Rastreamento Inteligente

NOVA LIMA - MG
2026

ERICK SILVA FERNANDES DE ARAÚJO

DOCUMENTAÇÃO INDIVIDUAL SOBRE OS TESTES UNITÁRIOS

Iveco Green Ledger - Sistema de Rastreamento Inteligente

Documentação de testes unitários apresentado ao Curso de Desenvolvimento de Sistemas do Serviço Nacional de Aprendizagem Industrial (SENAI), como requisito para a avaliação da Situação de Aprendizagem do projeto de Trabalho de Conclusão de Curso (TCC)

Orientador: Frederico Martins Aguiar

NOVA LIMA - MG

2026

1 - IDENTIFICAÇÃO DO PROJETO

Nome: Erick Silva Fernandes de Araújo (<https://github.com/erick190813>)

Turma: Desenvolvimento de Sistemas

Nome da Equipe: Green Ledger (https://github.com/NicolasOlim/Ivec_Green_Ledger.git)

Data de Entrega: 25/08/2026

Componente escolhido: ViewModel (Camada de Apresentação / Padrão MVVM)

Classes e Módulos: AnalisesViewModel, DashboardViewModel, FornecedorViewModel, MainViewModel, PecasViewModel, RastreabilidadeViewModel, RelatorioViewModel e ViewModelBase

Métodos Testados: AtualizarAsync(), AtualizarPegadaMediaAsync(), CanExecute() dos comandos de busca (ConsultarCnpjCommand, PesquisarVinCommand, SalvarFornecedorCommand, AdicionarPecaManualCommand), FazerLoginCommand.Execute() e OnPropertyChanged().

1.1 Justificativa da Escolha

A camada de ViewModel centraliza todas as regras de apresentação, validação de formulários, comandos executáveis (ICommand) e a comunicação com serviços e API's externas no padrão MVVM. A escolha por testar esse componente se deve ao fato de que as falhas na validação de entrada, falta de resiliência a quedas de rede ou inconsistências no estado da interface afetam diretamente a experiência do usuário final no ecossistema WPF. Garantir a cobertura unitária dessa camada garante que a lógica da interface funcione de forma estável e previsível antes mesmo da integração com telas gráficas ou bancos de dados.

2 RESPONSABILIDADE DO COMPONENTE

As ViewModels gerenciam o estado da interface, expõem propriedades com notificação de alteração (INotifyPropertyChanged) e fornecem comandos (ICommand) para a execução de ações assíncronas e consultas HTTP a API's externas.

- **O que o componentes faz:** Controla os dados exibidos nas telas, valida se as ações do usuários podem ser executadas e processa as respostas de serviços externos.
- **Problema que resolve:** Isola a lógica da interface gráfica do WPF, garantindo que botões e campos só fiquem habilitados com dados válidos e aplicando respostas de fallback caso API's externas estejam inacessíveis.

3 ESTRATÉGIA DE TESTES E COMPORTAMENTOS MAPEADOS

Os cenários foram definidos a partir das regras de negócio de cada tela, utilizando o manipulador MockHttpMessageHandler para interceptar requisições HTTP e simular as respostas das API's sem necessidade de conexão com a rede.

- **Cenários Válidos:** Formatação de valores, atualização de médias de emissão e preenchimento de dados de CNPJ válidos.
- **Cenários Inválidos:** Inserção de CNPJ com letras, senhas incorretas no login, ausência de categorias ESG ou ausência de vínculos de VIN e fornecedor.
- **Valores Limites e Exceções:** Validação de peso de peças, tamanhos e caracteres de VIN e tratamento de erros de rede.
- **Situações não testadas:** Renderização direta de elementos e conexão em tempo real com o banco.

4 TESTES DESENVOLVIDAS

Os testes foram executados na estrutura Arrange, Act e Assert:

Teste01:

CT06_CT07_CarregarTotalEmissoes_DeveFormatarEconomiaGeradaCorretamente

- **O que verifica:** Se a conversão do total de emissões e preço de carbono é formatada em milhares ("K") ou milhões ("M").

- **Arrange:** Instanciação do *MockHttpMessageHandler* configurado com respostas JSON simuladas para emissões e preço de carbono, criando o *HttpClient* com a URI mockada.
- **Act:** Chamada do método *viewModel.AtualizarAsync()*.
- **Assert:** Verificação do resultado por meio da asserção *Assert.Equal(formatacaoEsperada, viewModel.EconomiaGerada)*.

Teste02:

CT09_PrecoCarbonoFalha_DeveUsarFallbackCorretamente(AnalisesViewModelTestes)

- **O que verifica:** A utilização do valor padrão de fallback (\$150,0) quando a chamada HTTP para obter o preço do carbono retorna erro 500.
- **Arrange:** Configuração do mock de rede para responder com *HttpStatusCode.InternalServerError* na rota de preço do carbono.
- **Act:** Invocação do método *viewModel.AtualizarAsync()*.
- **Assert:** Confirmação por *Assert.Equal("R\$ 150,0K", viewModel.EconomiaGerada)*.

Teste03:

CT03_AtualizarPegadaMedia_ComFalhaDeRede_NaoDeveQuebrarAcesso(Dashboard ViewModelTestes)

- **O que verifica:** O comportamento do sistema ao enfrentar a ausência de conexão com a internet durante a consulta da pegada média.
- **Arrange:** Configuração do *SendAsyncFunc* no mock para lançar uma exceção *HttpRequestException("Sem internet")*.
- **Act:** Execução assíncrona de *viewModel.AtualizarPegadaMediaAsync()*.
- **Assert:** Verificação por *Assert.Equal("Indisponível", viewModel.PegadaMediaFormatada)*.

Teste04:

CT13_ConsultarCnpj_ComCnpjInvalido_NaoDevePermitirBusca(FornecedoresViewModelTestes)

- **O que verifica:** O bloqueio da busca de CNPJ caso a string informada contenha caracteres alfabéticos.
- **Arrange:** Atribuição da string "123AB/0001" à propriedade *CnpjBusca* do *FornecedorViewModel*.
- **Act:** Avaliação do método *viewModel.ConsultarCnpjCommand.CanExecute(null)*.
- **Assert:** *Assert.False(podeExecutar)* com mensagem indicando que a consulta não deve ser permitida fora do formato numérico.

Teste05:

CT20_CT21_ValidarVin_EntradasInvalidasELimites_DeveSerRejeitadas (RastreabilidadeViewModelTestes / MainViewModelTestes)

- **O que verifica:** A validação do campo de busca por VIN, rejeitando códigos de 16 ou 18 caracteres ou contendo as letras 'I', 'O' e 'Q', e aceitando o padrão correto de 17 caracteres.
- **Arrange:** Injeção parametrizada de strings via [Theory] e [InlineData] na propriedade.
- **Act:** Execução de *PesquisarVinCommand.CanExecute(null)*.
- **Assert:** Comparação do resultado retornado pelo comando com a regra de validação esperada.

Teste06:

CT17_CT18_AdicionarPeca_ValidacaoDePeso (PecasViewModelTestes)

- **O que verifica:** A aceitação de pesos positivos e iguais a zero (\$0.0\$ e \$65.50\$) e o bloqueio para pesos negativos (\$-5.0\$).

- **Arrange:** Atribuição do peso de teste e preenchimento dos vínculos obrigatórios (VIN, fornecedor e nome da peça).
- **Act:** Execução de *viewModel.AdicionarPecaManualCommand.CanExecute(null)*.
- **Assert:** *Assert.Equal(esperado, podeExecutar)*.

Teste07:

OnPropertyChanged_DeveDispararEvento_ComONomeDaPropriedadeCorreta (ViewModelBaseTestes)

- **O que verifica:** Se o método *OnPropertyChanged* dispara o evento *PropertyChanged* notificando o nome exato da propriedade modificada.
- **Arrange:** Instanciação da classe de teste *ViewModelMock* e subscrição ao evento *PropertyChanged*.
- **Act:** Alteração da propriedade *viewModel.MinhaPropriedade = "Teste Green Ledger"*.
- **Assert:** Confirmação por *Assert.Equal(nameof(ViewModelMock.MinhaPropriedade), propriedadeAlterada)*.

5 TÉCNICAS E RECURSOS APLICADOS

A tabela a seguir relaciona as bibliotecas, dependências do projeto e técnicas utilizadas na criação dos testes unitários:

Recurso/Ferramenta	Versão/Tipo	Aplicação nos Testes
xUnit	2.9.3	Framework de testes responsáveis pela estrutura das rotinas e pelas anotações Fact, Theory e InlineData.
Microsoft.NET.Test.Sdk	18.9.0	Pacote SDK para descoberta, complicação e suporte á

		execução no ecossistema .NET.
xunit.runner.visualstudio	4.0.0	Adaptador para integração e execução no Gerenciador de Testes do Visual Studio
Moq	4.20.72	Biblioteca utilizada para criação de mocks e simulação de objetos.
MockHttpMessageHandler	Classe Customizada	Herda de <i>HttpMessageHandler</i> para interceptar chamadas HTTP e injetar respostas simuladas via delegando <i>SendAsyncFunc</i> .

Para validar a entrada de dados (como CNPJ, VIN e peso de peças), os testes foram divididos simplesmente entre dados que o sistema deve aceitar e dados que deve rejeitar. Também foram testados os limites exatos das regras, garantindo que o sistema bloqueie pesos negativos enquanto aceita valores a partir do zero, e valide o tamanho exato do VIN (aceitando apenas 17).

6 RELATO DE DEFEITOS E CORREÇÕES

Durante a fase de desenvolvimento e execução dos testes unitários da classe `DashboardViewModelTestes`, identificou-se uma inconformidade no tratamento de exceções de rede.

- **Identificação:** Ocorreu durante a execução do teste *CT03_AtualizarPegadaMedia*.
- **Comportamento Observado:** A simulação de falha de conexão com a internet através da injeção de uma `HttpRequestException` resultava na interrupção do método `AtualizarPegadaMediaAsync()`, pois a exceção não estava sendo tratada na `ViewModel`.
- **Comportamento Esperado:** A `ViewModel` deveria capturar a exceção de rede e definir o texto "Indisponível" na propriedade `PegadaMediaFormatada` para exibição segura na interface gráfica.

- **Correção Aplicada:** Foi implementado um bloco try-catch capturando especificamente *HttpRequestException* dentro do método *AtualizarPegadaMediaAsync()* na classe *DashboardViewModel*. Com a alteração, a propriedade passou a receber a string amigável em caso de erro de conexão, garantindo a aprovação do teste e a estabilidade do sistema.

7 APRENDIZADO E CONTRIBUIÇÃO PARA O PROJETO

A criação dos testes unitários para a camada ViewModel permitiu validar a lógica de apresentação e as regras de entrada do projeto Green Ledger sem dependência direta de redes externas ou bancos de dados ativos. O uso da classe *MockHttpMessageHandler* garantiu a simulação de falhas de servidor (HTTP 500) e ausência de internet, assegurando que os mecanismos de fallback e tratamento defensivo funcionem conforme o esperado.

Em termos de aprendizado individual, a atividade proporcionou o domínio prático do padrão MVVM desacoplado, o uso do framework xUnit com testes parametrizados ([Theory]) e a estruturação de mocks HTTP no ambiente .NET.